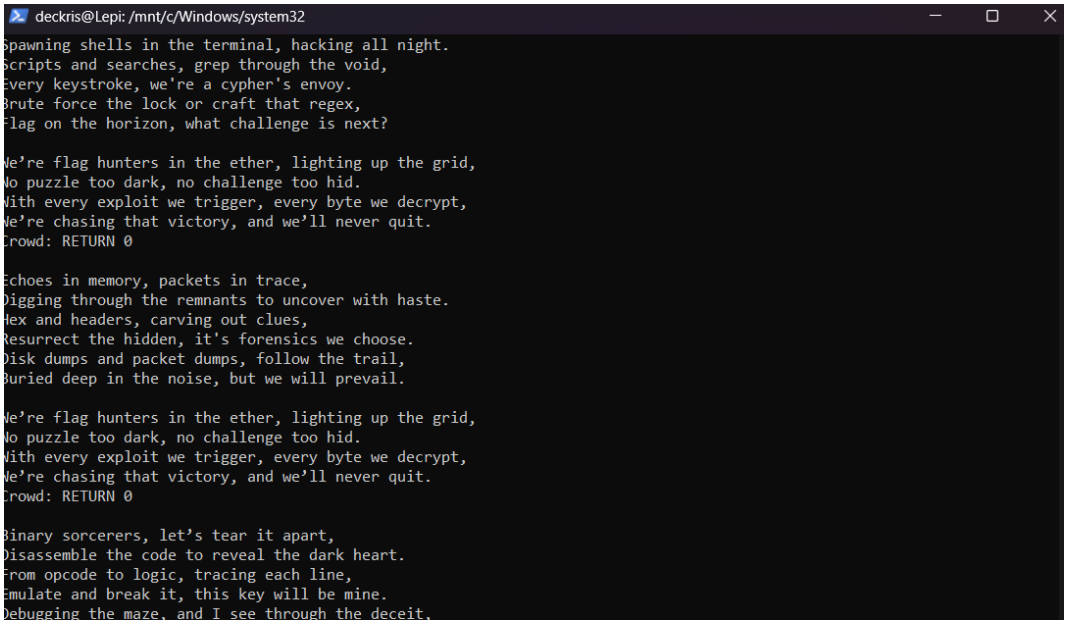
Homework 02

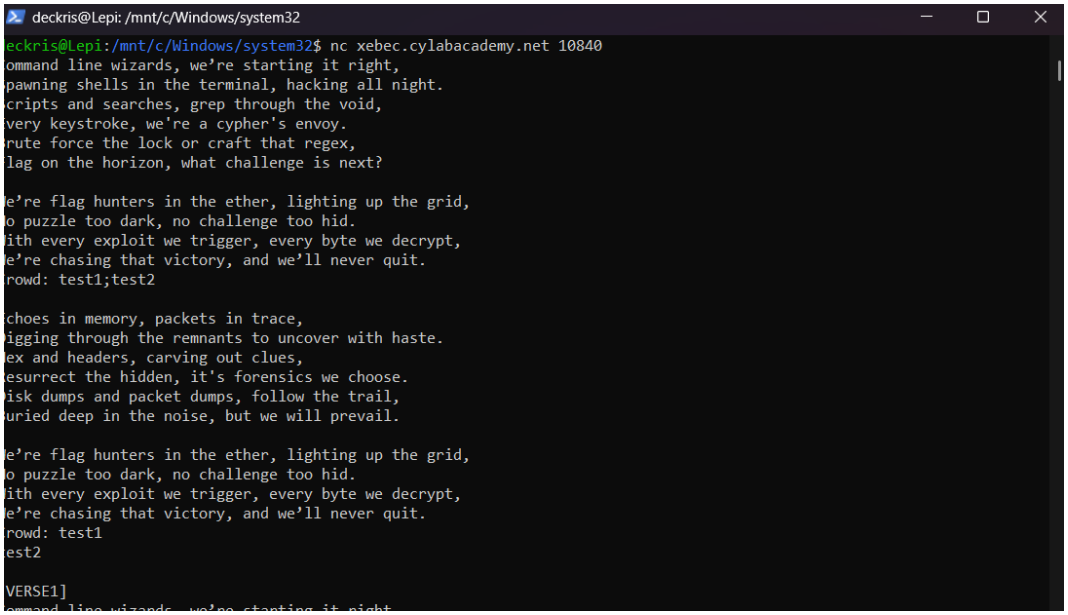
Eksplorasi Kategori CTF

1. Flag hunter (RE)

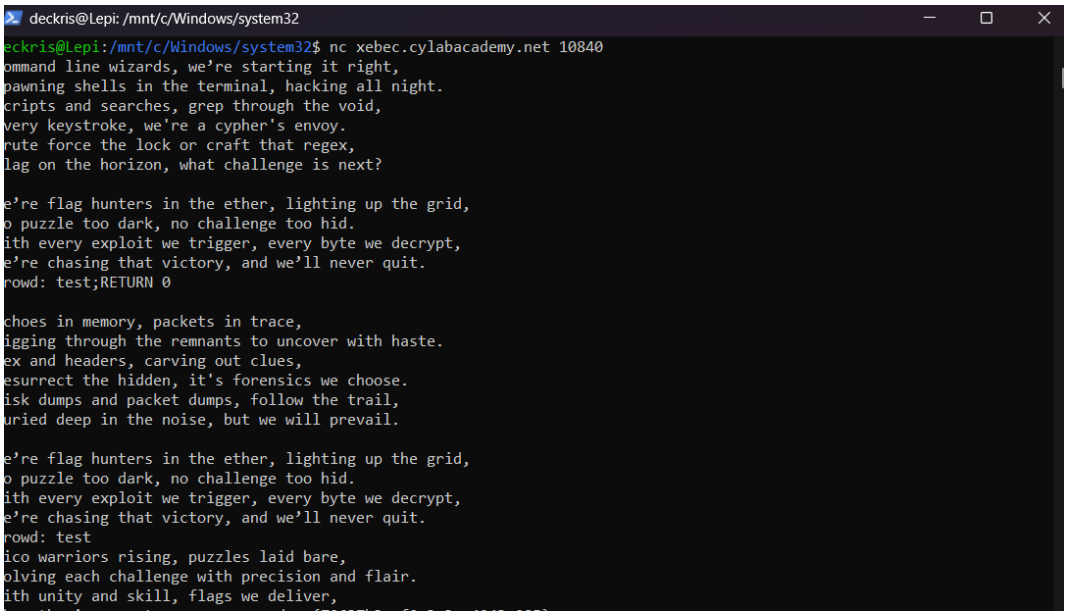
Di file lyric-reader.py, flag-nya disimpen di variabel secret_intro yang ada di baris pertama. Program ini pake *pointer* lip buat ngebaca lirik dan mecah instruksi lewat tanda titik koma (;). Celahnya ada pas program nerima input di bagian CROWD, soalnya input kita langsung menimpa array song_lines. Nah, karena ada pemisah ;, kita bisa ngakalin awalan Crowd: pake *payload* kaya test;RETURN 0. Bagian RETURN 0-nya bakal kebaca sama program buat memaksa nilai lip = 0, jadi alur eksekusi langsung mental balik ke baris pertama buat nyetak flag-nya.



Pas dicoba ngetik RETURN 0 polosan, program malah ngebaca itu sebagai Crowd: RETURN 0. Gara-gara kehalangan string Crowd: , regex RETURN <angka> gagal ketrigger, jadi lagunya lanjut terus tanpa mau melompat



Buat mastiin kalau ; beneran bisa mecah perintah, diinput test1;test2. Hasilnya Crowd: test1 diprint duluan terus test2 diprint di baris baru bawahnya. Ini ngebuktikin kalau .split(';') beneran ngebagi input jadi dua perintah terpisah

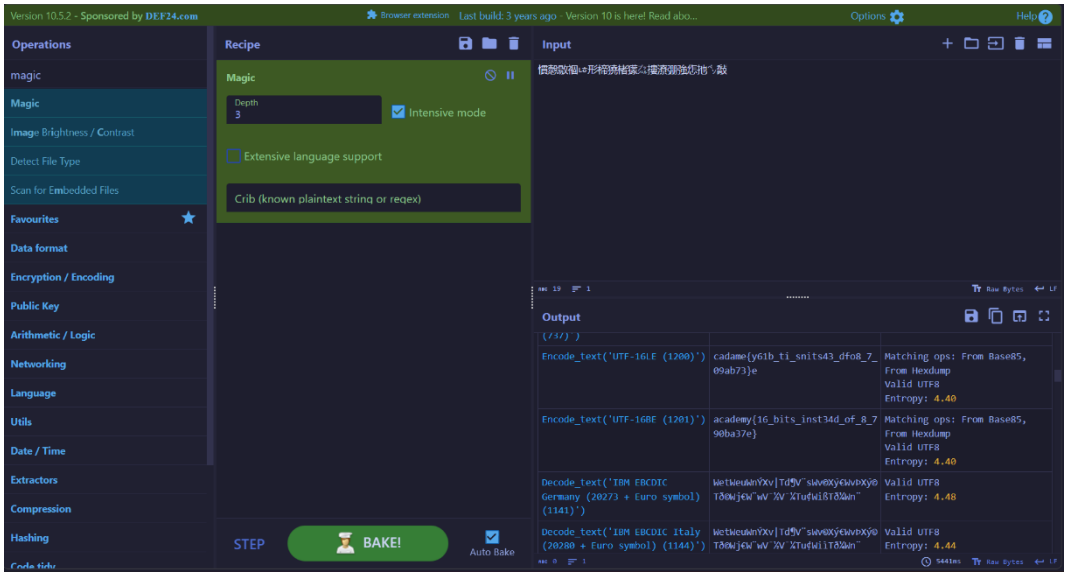


Triknya berhasil dipraktikkan pake payload test;RETURN 0. Bagian test diserap sama awalan Crowd: , sedangkan ;RETURN 0 dieksekusi sebagai instruksi baru yang langsung ngereset lip = 0. Program langsung loncat ke secret_intro di awal lagu dan nyetak flag-nya.

FLAG: **academy{70637h3r_f0r3v3r_4943a085}**

2. Transformation (RE)

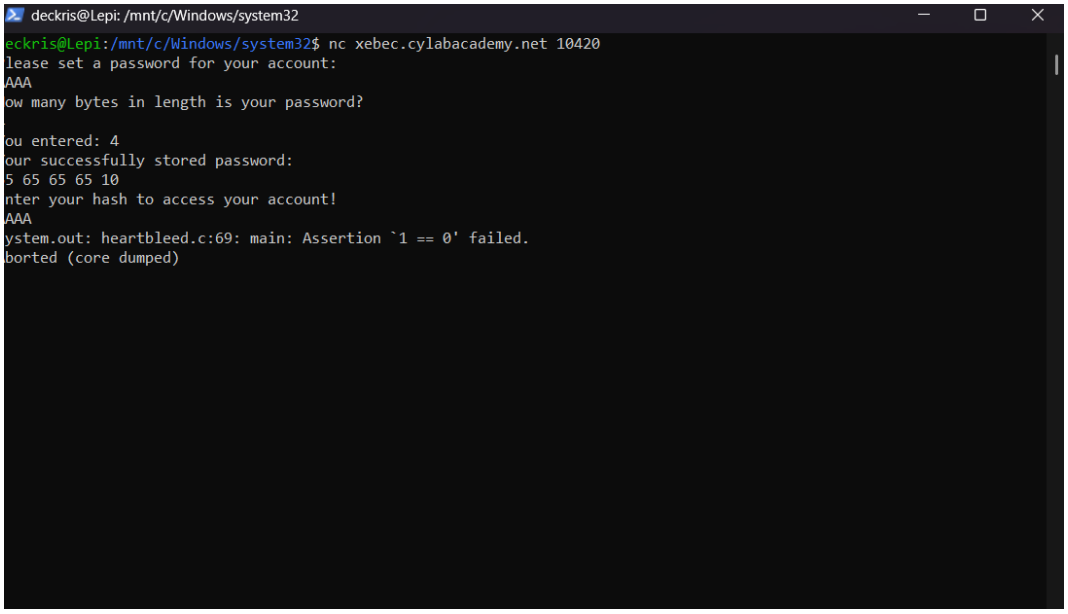
Di tantangan ini kita dikasih string aneh berisikan karakter Unicode berurutan: 慣殻敭禰ьŧ形楴獟楮㺚搂潦弸強悠扡ベゝ敲. Pas kita cek rumus enkripsinya, ternyata flag aslinya digabungin dua karakter dua karakter jadi satu huruf Unicode 16-bit. Tinggal direverse balik tiap karakter Unicode itu biar pecah lagi jadi dua huruf ASCII asal.



Karena di hint katanya ada decoders online, jadi kepake cyberchef lalu kita piliha magic dan langsung di bake. Lalu pada output kita cari flagnya yang relevan FLAG: **academy{16_bits_inst34d_of_8_790ba37e}**

3. Secure Password Database (RE)

Di soal ini kita dikasih akses ke service netcat dan sebuah file biner bernama system.out. Program ini bertindak sebagai database autentikasi password yang minta input password beserta panjang byte dari password tersebut.



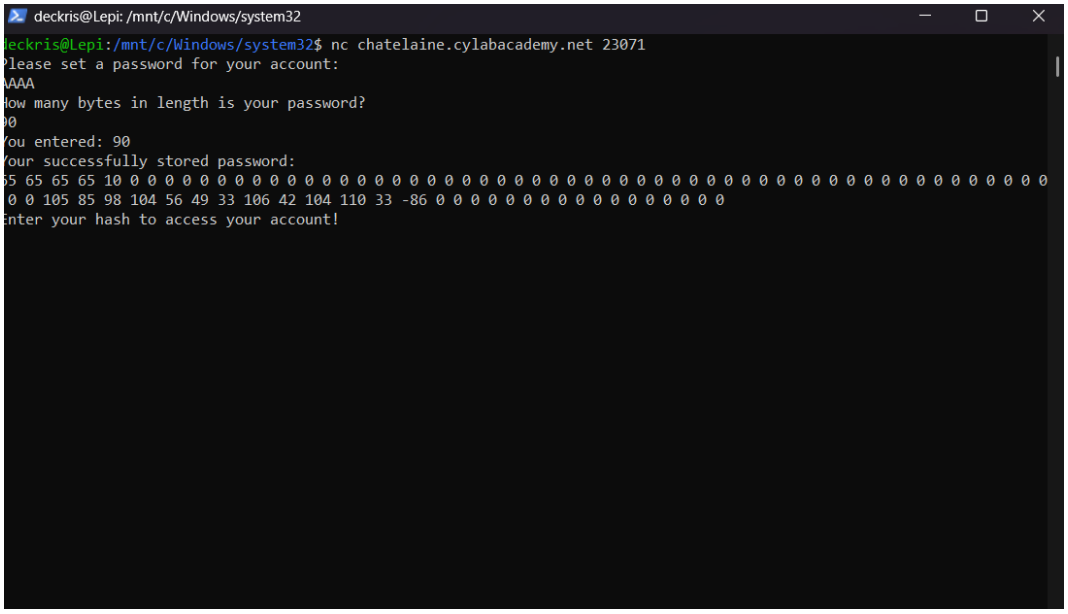
Pas program dijalankan via netcat, ada logika aneh di mana program menanyakan berapa panjang byte dari password yang kita masukkan. Jika kita memasukkan password pendek seperti AAAA dan masukin byte 4, dia bakal ada byte tambahan.

```
long hash(byte *param_1)

{
  byte *local_20;
  long local_10;

  local_10 = 0x1505;
  local_20 = param_1;
  while( true ) {
    if (*local_20 == 0) break;
    local_10 = (long)(int)(uint)*local_20 + local_10 * 0x21;
    local_20 = local_20 + 1;
  }
  return local_10;
}
```

Hasil dekompilasi file system.out memperlihatkan fungsi hash yang memproses password menggunakan algoritma berikut: Variabel awal local_10 diinisialisasi dengan nilai 0x1505 (desimal 5381). Setiap karakter dari password diproses dengan rumus: local_10 = (local_10 * 33) + ASCII(char) (di mana 0x21 hex sama dengan 33 desimal).


deckris@Lepi: /mnt/c/Windows/system32
deckris@Lepi:/mnt/c/Windows/system32$ nc chatelaine.cylabacademy.net 23071
Please set a password for your account:
AAAA
How many bytes in length is your password?
90
You entered: 90
Your successfully stored password:
65 65 65 65 10 0 0 0

4. Vault-door-training (RE)

Tantangan ini kita cuma dikasih satu file *source code* Java namanya VaultDoorTraining.java. Tugasnya cuma disuruh ngintip isi kodenya buat nyari password pintunya.

```java
import java.util.*;

class VaultDoorTraining {
    public static void main(String args[]) {
        VaultDoorTraining vaultDoor = new VaultDoorTraining();
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter vault password: ");
        String userInput = scanner.next();
    String input = userInput.substring("academy{".length(),userInput.length()-1);
    if (vaultDoor.checkPassword(input)) {
        System.out.println("Access granted.");
    } else {
        System.out.println("Access denied!");
    }
   }

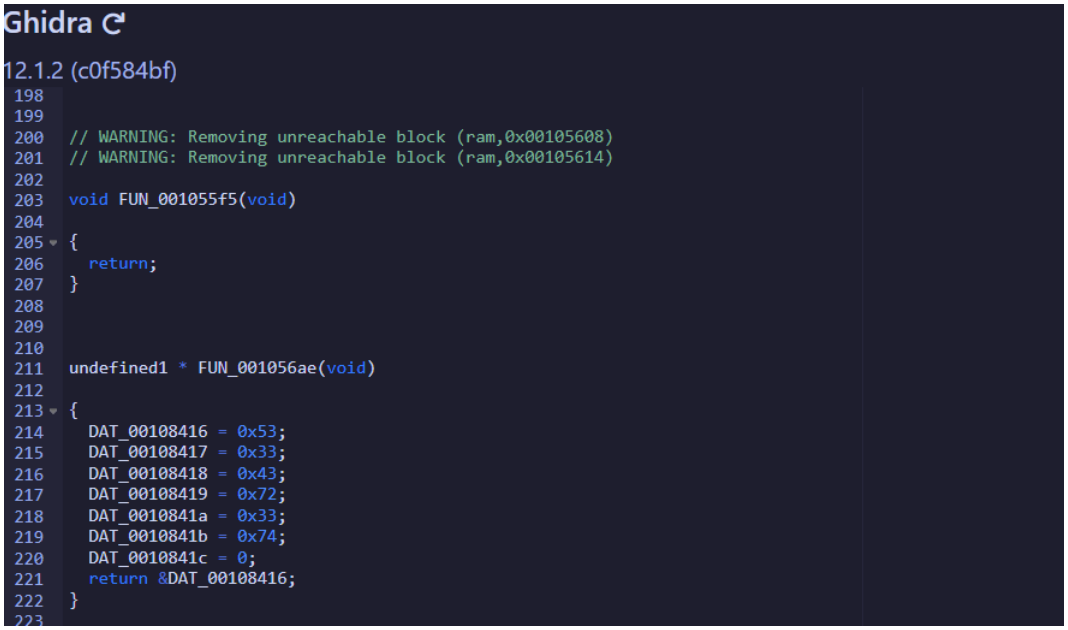
    // The password is below. Is it safe to put the password in the source code?
    // What if somebody stole our source code? Then they would know what our
    // password is. Hmm... I will think of some ways to improve the security
    // on the other doors.
    //
    // -Minion □#9567
    public boolean checkPassword(String password) {
        return password.equals("w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf");
    }
}
```

Pas file VaultDoorTraining.java dibuka pake terminal, kita langsung nemu fungsi checkPassword(). Di situ keliatan banget password-nya di-hardcode polosan tanpa dienkripsi sama sekali

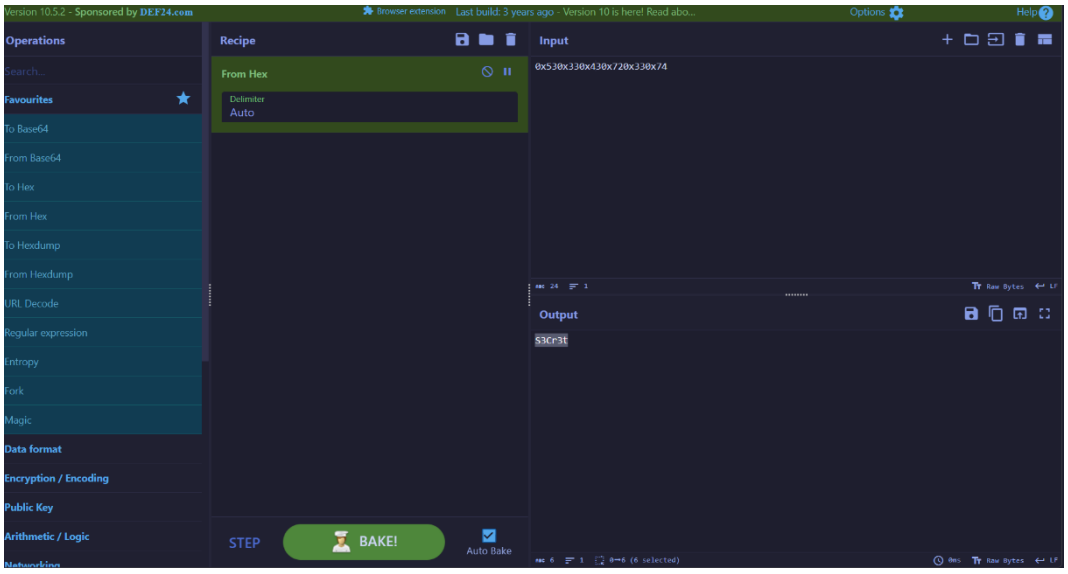
FLAG: **academy{w4rm1ng_Up_w1tH_jAv4_69bd4dc7adf}**

5. Hidden Cipher 1 (RE)

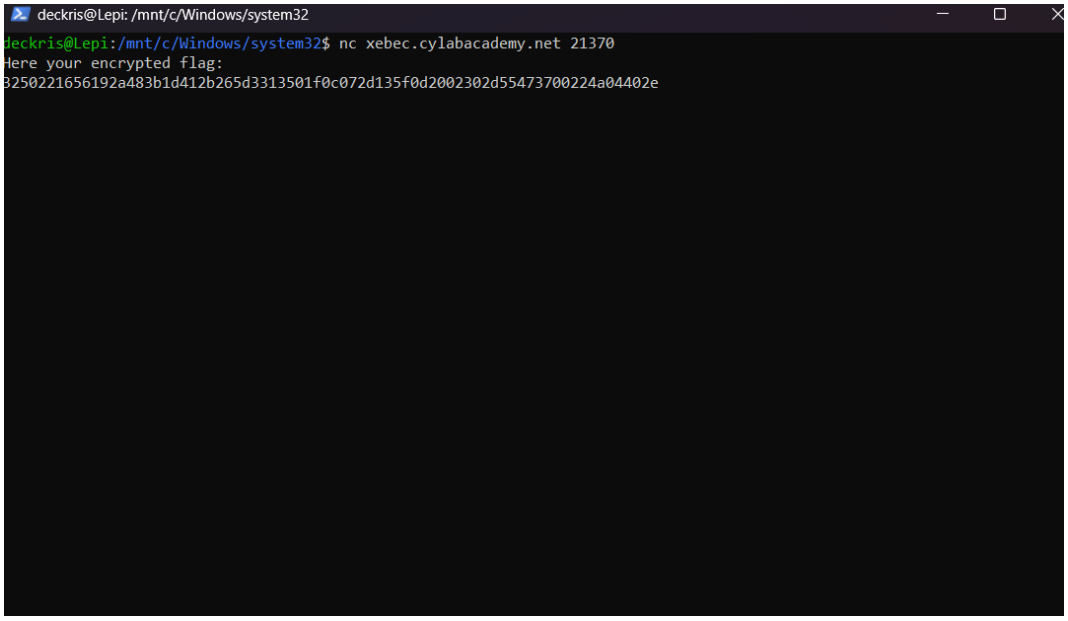
Di soal Hidden Cipher 1, kita dikasih satu file biner Linux. Kita disuruh mengartikan dan mengenkripsi flag yang ada di server lalu mencetaknya ke terminal. Tugas kita cukabongkar binernya, temukan kunci rahasia yang tersembunyi di dalamnya, lalu dekripsi string ciphertext yang kita dapatkan dari server.



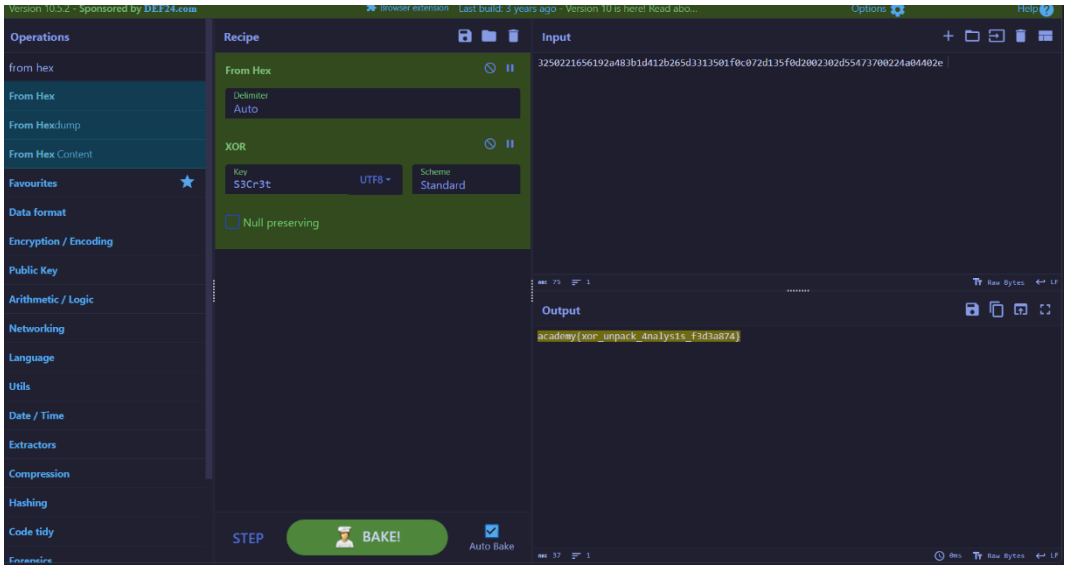
Setelah binernya berhasil diunpack dan didekompilasi pakai Ghidra, kita cek fungsi FUN_001056ae. Di fungsi itu keliatan ada deretan array heksadesimal yang didefinisikan langsung.



Pas digabungkan (0x533343723374) lalu dikonversi dari hex ke teks biasa di CyberChef pakai resep From Hex, kita dapet kunci rahasia yaitu **S3Cr3t.**



Selanjutnya kita tinggal konek ke servernya lewat netcat. Server bakal merespons dan ngasih string flag yang udah terenkripsi dalam format hex seperti di gambar.



Kita tempel ciphertext di CyberChef pakai susunan resep From Hex + XOR dengan isi kunci S3Cr3t buat dapetin flag-nya.

FLAG: **academy{xor_unpack_4nalys1s_f3d3a874}**

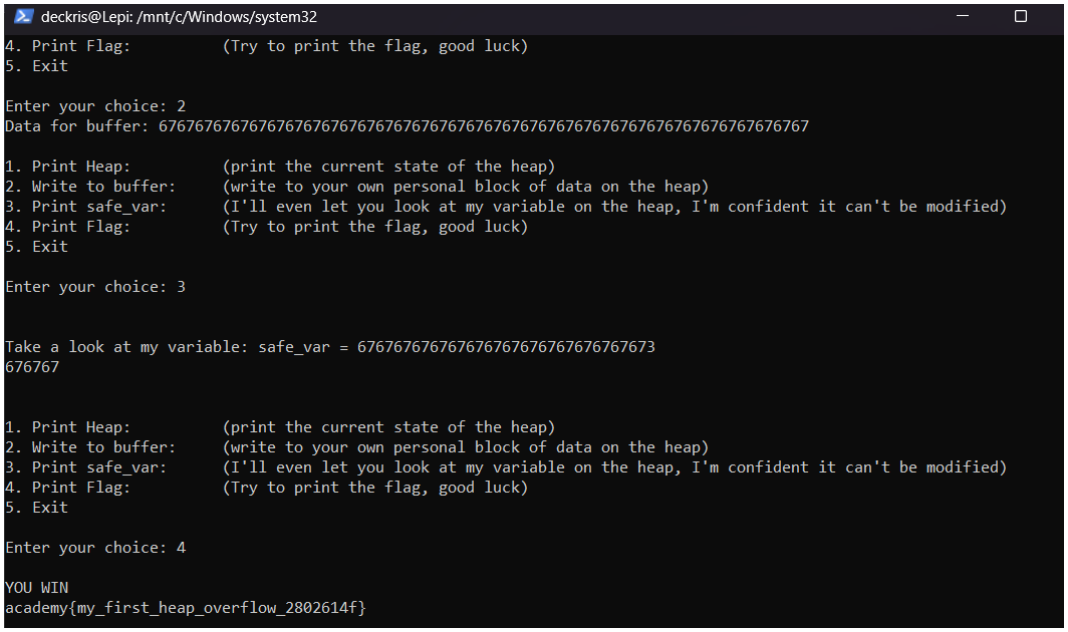
6. Heap 0 (BE)

Pada tantangan heap 0, kita diberikan sebuah biner beserta source code C yang mensimulasikan alokasi data di dalam memori heap. Program menyimpan dua

variabel utama di heap, yaitu buffer input dan sebuah variabel pengaman bernama safe_var yang diisi teks bico. Tugas kita adalah ngubah isi safe_var agar tidak lagi bernilai bico demi mendapatkan flag.

```
void check_win() {
    if (strcmp(safe_var, "bico") != 0) {
        printf("\nYOU WIN\n");
```

Kita periksa source code fungsi check_win, ada syarat pembandingan string menggunakan strcmp. Selama safe_var masih isi string bico, program gak akan ngasi flag.

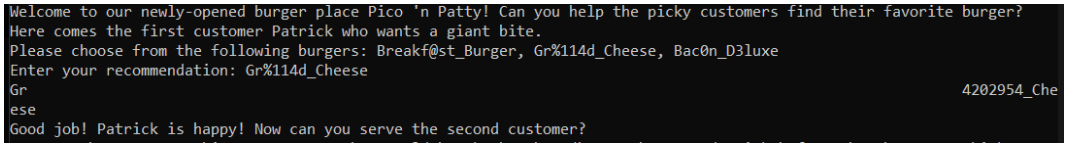


Kita pilih menu no 2 dan masukin input yang acak yang panjang untuk overflow. Pilih menu 3 buat memastikan save_var tidak bernilai bico dan langsung mencetak flag.

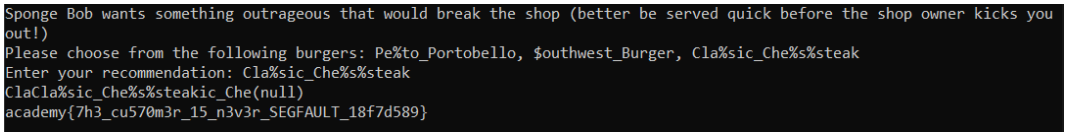
FLAG: **academy{my_first_heap_overflow_2802614f}**

7. Format string 0 (BE)

Di soal format string 0, kita disimulasikan sebagai pelayan di kedai burger. Tugas kita cuma melayani dua pelanggan dengan cara memilih menu burger yang ada.



Patrick minta burger porsi raksasa. Kita perhatiin opsi menu yang disediakan, pilihan GrilL3d Ch33s3 mengandung format specifier %114d. Format ini bakal memaksa fungsi printf buat mencetak angka desimal sepanjang 114 digit, sampai bikin programnya overflow.



SpongeBob minta sesuatu yang ekstrem buat merusak sistem. Pilihan ClaSsIc_Ch33s3_StEaK mengandung format specifier %s yang bertugas membaca string dari alamat memori di stack, fungsi printf bakal membaca nilai *pointer* di *stack* yang bakal ngarah ke flag.

FLAG: **academy{7h3_cu570m3r_15_n3v3r_SEGFAULT_18f7d589}**